

开源 AI Agent CLI — 功能分析与对比报告

v1.0 @ 02.2026

Authors: Andrei Leman & Claude Opus 4.6

生成日期：2026-02-26 | 数据来源：42 个知识库条目，涵盖 4 个已爬取的代码仓库及网络调研
知识库来源：OpenCode (262)、Crush (263)、awesome-opencode (264)、Codex (265)

1. 概述

2026 年初，AI 编程 Agent CLI 领域已发展为一个竞争激烈的生态系统，约有 15 款值得关注的工具。主要趋势如下：

- Go 和 Rust 主导新一代 CLI 实现（OpenCode、Crush、Codex-rs、Goose），在性能方面取代了 Python/TypeScript
- **MCP（Model Context Protocol）**已成为通用的可扩展性标准——所有主流工具均已支持或计划支持
- 多模型提供商支持已成为基本要求——各工具通过抽象层支持 7 至 12 个以上的 LLM 提供商
- LSP 集成是区分专业工具与玩具的关键——仅 OpenCode、Crush 和 Zed 具备深度 LSP 支持
- 沙箱机制是新的技术前沿——Codex 凭借 Seatbelt/Landlock 领先，其他工具依赖权限提示
- Agent 即服务器模式正在兴起——Codex 可作为 MCP 服务器运行，实现元 Agent 架构
- TUI 框架趋同于 Bubble Tea（Go）和 Ratatui（Rust），用于构建丰富的终端界面

市场定位（2026 年 2 月）

层级	工具	Star 数
领先	Gemini CLI (~95K)、Zed (~76K)、Cline (~58K)、Codex (~49K)、Claude Code (~41K)、Aider (~41K)	50K+
强势	Continue (~32K)、Goose (~31K)、OpenCode (~30K)、Warp (~26K)	25-50K
新兴	Qwen Code (~15K)、Plandex (~15K)、Crush (~14K)、Codebuff (~3K)	<15K

2. 各工具深度分析

2.1 OpenCode (opencode-ai/opencode)

概述：基于 Go 的终端 AI 编程助手，采用 Bubble Tea TUI。约 29.7K Star，Apache-2.0 许可证。

架构：

- cmd/ → CLI 入口
- internal/app → 服务编排
- internal/llm → Agent 循环 + 提供商 + 工具 + 提示词
- internal/tui → Bubble Tea 界面
- internal/lsp → Language Server Protocol 集成
- internal/db → 基于 sqlc 的 SQLite 持久化
- 配置：基于 Viper，~/.opencode.json 或 ~/.opencode.json

Agent 循环：单线程 ReAct 循环。processGeneration() 进入工具使用循环，按顺序执行工具并进行权限检查。当完成原因为 ToolUse 时循环继续。通过以 sessionId 为键的 sync.Map 实现取消操作。

工具 (12+)：

工具	描述
bash	持久化 Shell 会话，默认 60 秒 / 最大 600 秒超时。安全命令跳过权限检查。禁止：curl、wget、nc、telnet、浏览器
edit	基于行范围的文本替换，带冲突检测
write	文件创建/更新，自动创建目录，检查修改时间与上次读取的一致性
view	文件读取，支持行范围
patch	统一 diff 补丁，带模糊检测（模糊度 > 3 时拒绝），原子化应用
glob	文件模式匹配
grep	双策略：优先使用 ripgrep，回退到正则表达式。100 个文件限制，200 个匹配上限
ls	目录列表
fetch	HTTP 请求
sourcegraph	通过 Sourcegraph API 进行跨仓库代码搜索
agent	子 Agent 生成（只读工具子集：glob、grep、ls、sourcegraph、view）
diagnostics	LSP 诊断（当 LSP 可用时）

****提供商支持：****7+ 个提供商——OpenAI、Anthropic、Google、Azure、AWS Bedrock、Groq、OpenRouter，以及 OpenAI 兼容 API。

****MCP：****客户端 MCP 支持，用于连接外部工具服务器。

****LSP 集成：****通过 `internal/lsp` 实现深度集成。诊断工具查询语言服务器。Write 和 patch 工具在文件更改后等待 LSP 诊断以捕获语法错误。

****TUI：****Bubble Tea 框架配合 Lip Gloss 样式。多面板布局。

****会话管理：****基于 `sqlc` 的 SQLite 存储。会话持久化、消息历史、标题自动生成。

权限系统：`internal/permission` 包。安全的只读命令（`git status`、`go test`、`ls`）跳过审批。文件操作要求先读后编辑。权限拒绝会停止剩余工具的执行。

独特功能：

- Sourcegraph 集成，支持跨仓库搜索
- 具有受限只读工具集的子 Agent
- LSP 诊断集成到 write/patch 工作流中
- 基于 Viper 的分层配置（用户 + 项目）

2.2 Crush (charmbracelet/crush)

****概述：****由 Charmbracelet 开发的终端 AI 编程 Agent。Go 1.26，约 14.4K Star。基于 Fantasy 框架（`charm.land/fantasy`）构建。

架构：



- agent.go — 每个会话的 Agent 循环，消息队列，自动摘要
- tools/*.go — 20+ 内置工具，通过 Fantasy AgentTool 接口实现
- tools/mcp/ — 完整 MCP 支持（stdio/HTTP/SSE 传输）
- hyper/provider.go — Charmbracelet 自有的 LLM 代理（hyper.charm.land）
- prompt/ — Go text/template 系统，编译时嵌入模板

工具 (20+)：

工具	描述
bash	Shell 执行，支持后台运行。超过 60 秒自动转为后台。黑名单：curl、wget、ssh、sudo、apt、brew 等
edit	单文件编辑，强制先读后编辑，修改时间检查
multi_edit	单文件批量顺序编辑
view	文件读取，查看时附加 LSP 诊断信息
glob	文件模式匹配
grep	内容搜索
ls	目录列表
write	文件创建
fetch	HTTP 请求（5MB 限制，600 秒超时，仅限 http/https）
download	文件下载
web_search	DuckDuckGo 搜索（无需 API 密钥）
sourcegraph	通过 Sourcegraph GraphQL API 进行跨仓库搜索（最多 20 条结果）
agent	子 Agent 生成，用于委派任务
agentic_fetch	专用子 Agent，用于网络调研，拥有独立工具集
diagnostics	LSP 诊断（文件级 + 项目级，按严重程度排序）
references	LSP FindReferences，带 grep 预过滤，支持限定符号
lsp_restart	重启特定或所有 LSP 客户端
job_output	获取后台 Shell 的 stdout/stderr
job_kill	终止后台 Shell
todos	内置任务跟踪（pending/in_progress/completed）
list_mcp_resources	列出 MCP 服务器资源
read_mcp_resource	读取特定 MCP 资源

****提供商支持：****通过 Fantasy 框架实现多提供商支持——OpenAI、Anthropic、Google、Azure、Bedrock，以及 Hyper（Charmbracelet 自有代理，hyper.charm.land）。Hyper 支持 JSON 和流式模式、带 Retry-After 的重试机制、额度跟踪。

MCP 集成（完整）：

- 三种传输方式：stdio、HTTP（StreamableClient）、SSE
- 四种状态：Disabled → Starting → Connected → Error（通过 pubsub 发布）
- 完整支持：工具、资源、提示词
- 工具命名：mcp_[serverName]_[toolName]
- 每次 MCP 工具执行前进行权限检查
- 按模型验证图像/媒体能力

LSP 集成（深度）：

- lsp.Manager 管理多个语言服务器客户端
- 诊断：按严重程度排序，每类上限 10 条，支持文件级 + 项目级
- 引用：grep 预过滤 → LSP FindReferences，支持限定符号 (::、.、\\)
- LSP 重启支持并发 goroutine
- 在 crush.json 中配置 (Go、TypeScript、Nix 及任意 LSP 服务器)

****TUI:** ****完整 Charm 技术栈——Bubble Tea (框架)、Lip Gloss (样式)、Glamour (Markdown)、Bubbles (组件)。**

****会话管理:** ****Coordinator 管理会话。自动摘要、标题生成。基于会话的权限上下文。**

****权限系统:** ****多层设计:**

- Bash: 命令黑名单 + 参数拦截 + 安全白名单
- 文件: 先读后编辑、修改时间检查、符号链接解析、工作目录外权限控制
- MCP: 逐工具权限控制、禁用工具过滤
- 全局: --yolo 标志跳过所有检查, crush.json 白名单

****循环检测:** ****对滑动窗口 (10 步) 内的 (工具名 + 输入 + 输出) 进行 SHA-256 哈希。任一签名重复 5 次以上即触发。**

****提示词系统:** ****Go text/template 配合编译时嵌入。三个模板: coder (主模板)、task (子 Agent)、initialize。注入 git 状态、分支、最近提交、技能 XML、上下文文件。**

****技能系统:** ****从 ~/.config/crush/skills/ 及自定义路径发现可用包。序列化为 XML 注入提示词。**

独特功能:

1. Hyper 提供商 (Charmbracelet 的 LLM 代理)
2. Sourcegraph 跨仓库搜索
3. Agentic fetch (专用网络调研子 Agent)
4. 后台任务, 支持自动转后台 (60 秒阈值)
5. 内置待办事项跟踪
6. DuckDuckGo 搜索 (无需 API 密钥)
7. 多文件编辑工具
8. Fantasy 框架 (charm.land/fantasy)
9. .crushignore 支持
10. [AGENTS.md](#) 项目上下文

2.3 OpenAI Codex CLI (openai/codex)

****概述:** ****双层架构——旧版 TypeScript CLI (已被取代) 和当前 Rust CLI。约 49.3K Star, Apache-2.0 许可证。**

架构:

```
Rust 工作空间 (codex-rs/):
core/      - 业务逻辑库
exec/      - 无头 CLI
tui/       - 基于 Ratatui 的 TUI
cli/       - 多功能二进制文件
app-server-protocol/ - JSON-RPC 协议定义
```

多种部署形态: CLI、IDE 扩展 (VS Code/Cursor/Windsurf)、Codex App (macOS 桌面应用)、Codex Web (chatgpt.com/codex)。通过 npm (@openai/codex)、Homebrew、GitHub Release 二进制文件分发。

Agent 循环: AgentLoop 中的 ReAct 风格"思考-行动-观察"循环。结构化输出约束防止幻觉。自愈机制: 将 stderr 反馈为上下文。Token 优化: 截断、滑动窗口 (3-5 次交互)、批处理。

工具 (精简但强大):

工具	描述
apply_patch	通过 codex-arg0 crate 实现的虚拟 CLI，统一 diff 补丁
local_shell_call	沙箱化 Shell，支持命令数组、工作目录、环境变量、超时
web_search_call	网络搜索
function_call	标准 OpenAI 函数调用
custom_tool_call	通过 DynamicToolSpec 实现的 MCP/动态工具

****提供商支持 (10+): ****OpenAI (默认, o4-mini)、OpenRouter、Azure、Gemini、Ollama、Mistral、DeepSeek、xAI、Groq、ArceeAI, 以及任何 OpenAI 兼容 API。使用 Responses API。

MCP (双向):

- 客户端: 从 config.toml 连接服务器
- 服务器: codex mcp-server (实验性) ——允许其他 Agent 将 Codex 作为工具使用
- 管理: codex mcp add/list/get/remove
- 支持 MCP 服务器的 OAuth 认证

****LSP: ****无原生 LSP 集成。

****TUI: ****基于 Ratatui 的终端界面 (Rust)。

****会话管理: ****SQLite 状态数据库 (CODEX_SQLITE_HOME)。Ghost commit 用于不可见的仓库快照。支持加密内容的压缩。Plan 模式支持独立的推理力度配置。codex exec --ephemeral 用于无持久化运行。

沙箱模型 (业界最佳):

- macOS: Apple Seatbelt (sandbox-exec) ——只读沙箱, 可配置可写根目录, 网络完全阻断
- Linux: 通过 codex-arg0 crate 自重执行实现 Landlock LSM; 推荐使用 Docker 以获得更强隔离
- Windows: 仅支持 WSL2
- 三种模式: 只读 (默认)、工作区可写、危险-完全访问
- 全自动模式 = 禁用网络 + 目录限制

权限系统: AskForApproval 枚举: untrusted、on-failure、on-request、never、reject。

****应用-服务器协议: ****前端与核心后端之间的 JSON-RPC。关键类型: InitializeParams、CommandExecParams、ConfigRead/Write、FuzzyFileSearch、ReviewStartParams。枚举: SandboxMode、AskForApproval、ReasoningEffort (none→xhigh)、Personality (none/friendly/pragmatic)。

配置: ~/.codex/ 下的 TOML 格式。分层: 用户 → 项目。AGENTS.md 层级: ~/.codex/AGENTS.md → 仓库根目录 → 当前工作目录。支持外部 Agent 配置检测/导入。

****技能系统: ****基于文件系统, 位于 .codex/skills/<name>/。SKILL.md 带 YAML 前置元数据。内置技能: babysit-pr (自主 PR 监控)、test-tui。Codex App 将技能扩展为 Automations (定时后台任务)。

独特功能:

1. Ghost commit (不可见的仓库快照)
2. Babysit-pr 技能 (自主 DevOps Agent, 支持 CI 失败分类)
3. Codex 作为 MCP 服务器 (元 Agent 架构)
4. 外部 Agent 配置迁移
5. 个性系统 (none/friendly/pragmatic)
6. 内置代码审查 (uncommittedChanges/baseBranch/commit/自定义目标)
7. Plan 模式, 支持可配置的推理力度
8. Rust 原生性能
9. 业界最佳沙箱机制 (Seatbelt + Landlock)
10. 多种部署形态 (CLI、IDE、App、Web)

2.4 Claude Code (Anthropic)

概述：Anthropic 官方的 Claude CLI。TypeScript，约 40.8K Star。AI 编程 Agent 的参考实现。

架构：Node.js CLI，具备 Hooks 系统、MCP 集成、技能系统，以及通过 Task 工具生成子 Agent。

工具 (10+)：Read、Edit、Write、Glob、Grep、Bash、Task（子 Agent）、WebFetch、WebSearch、NotebookEdit、AskUserQuestion、EnterPlanMode，以及 MCP 工具。

提供商支持：仅支持 Anthropic 模型（Claude Opus 4.6、Sonnet 4.6、Haiku 4.5）。模型路由：Opus 用于架构设计，Sonnet 用于功能开发，Haiku 用于快速任务。

MCP：完整客户端支持。工具、资源、提示词。在 settings.json 中配置。

LSP：无原生 LSP（作为扩展使用时依赖 IDE 集成）。

TUI：终端聊天界面，支持 Markdown 渲染。

会话管理：JSONL 转录文件。在约 120K Token 时进行上下文压缩。Pre-compact Hook 用于保持连续性。

权限系统：权限模式，支持工具级审批。Hook 可拦截并允许/拒绝/询问。CLAUDE.md 用于项目指令。

Hooks 系统 (14 个事件)：SessionStart、UserPromptSubmit、PreToolUse、PermissionRequest、PostToolUse、PostToolUseFailure、Notification、SubagentStart、SubagentStop、Stop、TeammateIdle、TaskCompleted、PreCompact、SessionEnd。Hook 可注入上下文、阻止操作、修改权限。

独特功能：

1. 最成熟的 Hook/生命周期系统（14 个事件）
2. 具有隔离上下文的子 Agent 生成（Task 工具）
3. 技能系统（用户可调用的斜杠命令）
4. CLAUDE.md 项目指令（层级化）
5. 带 pre-compact Hook 的上下文压缩
6. 权限模式，支持细粒度工具级控制
7. Notebook 编辑支持
8. Plan 模式，支持用户审批流程

2.5 Gemini CLI (Google)

概述：Google 的终端 AI Agent。TypeScript，约 95.7K Star（该类别最高）。Apache-2.0 许可证。

关键规格：

- 100 万 Token 上下文窗口（目前最大）
- Gemini 2.5 Pro / Gemini Flash（仅 Google 模型）
- MCP 支持（本地和远程服务器）
- 对话检查点（保存/恢复）
- 受信任文件夹安全系统
- GEMINI.md 项目配置
- 多模态输入（草图、PDF）
- Google 搜索增强
- 免费层：60 次请求/分钟，1K 次请求/天

独特功能：超大上下文窗口、慷慨的免费层、Google 搜索集成、多模态输入。

2.6 Aider

概述：AI 结对编程 CLI。Python，约 40.9K Star。Apache-2.0 许可证。仓库地图（repo-map）概念的先驱。

关键规格：

- 基于 Tree-sitter 的仓库地图，用于代码库理解
- 自动 git 提交作为检查点
- 语音转代码
- IDE 监视模式
- 代码检查/测试自动修复
- 支持 100+ 种语言
- 编辑格式：whole/diff/udiff
- Architect 模式（双模型工作流）
- SWE-bench 排行榜领先
- 无原生 MCP（社区分支 mcpm-aider 存在）
- 无原生 LSP（使用 Tree-sitter）
- 模型支持：Claude、DeepSeek、GPT-4o、o1、o3-mini，通过 Ollama 支持几乎任何 LLM

独特功能：开创了仓库地图、git 会话管理、Architect 模式、语音转代码、最广泛的语言支持。

2.7 Goose (Block/Square)

概述：可扩展 AI Agent。Rust (59%) + TypeScript (33%)，约 31.2K Star。Apache-2.0 许可证。

关键规格：

- 扩展即 MCP 服务器（MCP 一等公民架构）
- CLI + 桌面 GUI 应用
- 支持任意 LLM 提供商，可同时配置多个
- Recipes 系统，用于可组合的工作流
- Mentor 模式，用于学习
- 113 个版本（v1.25.0）
- 基于会话的对话历史

独特功能：扩展即 MCP 服务器架构、CLI + 桌面双形态、Recipes、Mentor 模式。

2.8 其他值得关注的工具

Cline (~58.4K Star) — VS Code 扩展。自主多步执行、通过 Playwright 实现浏览器自动化、Computer Use 能力、工作区检查点、按需创建 MCP 工具。分支项目：Roo Code、Kilo Code。

Continue.dev (~31.5K Star) — VS Code/JetBrains 扩展 + CLI。AI 检查作为 GitHub 状态检查、检查即代码（`.continue/checks/`）、MCP Registry 集成。

Warp (~26K Star) — GPU 加速终端替代品。多 Agent 编排（Oz Agent），支持 CLI Agent（Claude Code、Codex、Gemini CLI）。企业版 \$200/月。

Zed (~75.9K Star) — Rust 编辑器，具备 Agent Panel、Zeta 自动补全模型、ACP（Agent Client Protocol）用于第三方 Agent、多人协作编辑。作为 IDE 具备完整 LSP 支持。

Amp/Sourcegraph — 闭源。多模型自动路由、Deep 模式、Oracle/Librarian 子 Agent、Sourcegraph 代码智能底座、共享线程。

Codebuff (~2.9K Star) — 多 Agent 架构（File Picker → Planner → Editor → Reviewer）、[knowledge.md](#) 记忆、可嵌入 SDK。声称评测得分 61%，优于 Claude Code 的 53%。

Plandex (~14.6K Star) — Go 语言，超大上下文（20M+ Token），沙箱执行。

Qwen Code (~14.9K Star) — Gemini CLI 分支，针对 Qwen3-Coder 优化。

3. 功能对比矩阵

核心功能

功能	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
语言	Go	Go	Rust+TS	TypeScript	TypeScript	Python	Rust+TS
Star 数	~30K	~14K	~49K	~41K	~96K	~41K	~31K
许可证	Apache-2.0	Apache-2.0	Apache-2.0	专有	Apache-2.0	Apache-2.0	Apache-2.0
TUI 框架	Bubble Tea	Bubble Tea	Ratatui	自定义	自定义	终端	CLI+桌面

Agent 循环与工具

功能	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
Agent 循环	ReAct 单线程	ReAct 单线程	ReAct 单线程	ReAct 单线程	ReAct 单线程	ReAct 单线程	ReAct 单线程
内置工具	12+	20+	2 核心 + 3	10+	~8	~6	基于 MCP
子 Agent	是（只读）	是（agentic fetch）	否	是（Task 工具）	否	否（Architect 模式）	否
后台任务	否	是（自动转后台 60 秒）	否	否	否	否	否
循环检测	否	是（SHA-256 窗口）	否	是（Hook）	否	否	否
待办跟踪	否	是（内置）	否	是（TaskCreate）	否	否	否

提供商与模型支持

功能	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
提供商	7+	多个 + Hyper	10+	仅 Anthropic	仅 Google	任意 LLM	任意 LLM
本地模型	通过 OpenAI 兼容	通过提供商	Ollama	否	否	Ollama	Ollama
模型路由	否	否	否	是（Opus/Sonnet/Haiku）	否	Architect 模式	多配置
自有代理	否	Hyper (charm.land)	否	否	否	否	否

MCP 与 LSP

功能	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
MCP 客户端	是	是（完整）	是	是	是	否	是（一等公民）
MCP 服务器	否	否	是（实验性）	否	否	否	否
MCP 传输	stdio	stdio/HTTP/SSE	stdio	stdio	stdio	—	stdio
MCP 资源	否	是	否	是	否	—	是
MCP 提示词	否	是	否	是	否	—	否
LSP 诊断	是	是（深度）	否	否	否	否	否
LSP 引用	否	是	否	否	否	否	否
LSP 重启	否	是	否	否	否	否	否

会话与配置

功能	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
持久化	SQLite	会话存储	SQLite	JSONL 文件	检查点	Git 提交	会话文件
配置格式	JSON	JSON (crush.json)	TOML	JSON	GEMINI.md	CLI 标志/YAML	YAML
配置层级	用户 + 项目	用户 + 项目	用户 + 项目	全局 + 项目	项目	CLI + 配置	项目
项目文件	.opencode.json	AGENTS.md	AGENTS.md	CLAUDE.md	GEMINI.md	.aider*	GOOSE.md
上下文窗口	标准	自动摘要	滑动窗口	120K + 压缩	100 万 Token	仓库地图	标准
自动摘要	是	是	是（压缩）	是（压缩）	否 (超大上下文)	否	否

安全与权限

功能	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
权限系统	是	是 (多层)	是（沙箱）	是（模式 +Hook）	受信任文件夹	否	治理
沙箱	否	否	是 (Seatbelt/Landlock)	否	是	否	否
命令黑名单	是 (curl,wget,nc)	是 (广泛)	不适用（沙箱化）	否（基于 Hook）	否	否	否
先读后编辑	是	是	不适用	是	否	否	否
跳过全部标志	否	--yolo	danger-full-access	否	否	否	否
修改时间检查	是	是	否	否	否	否	否

可扩展性

功能	OpenCode	Crush	Codex	Claude Code	Gemini CLI	Aider	Goose
技能/插件	否	是 (技能目录)	是 (SKILL.md)	是 (斜杠命令)	否	否	Recipes
Hook/ 生命周期	否	否	否	是 (14 个事件)	否	否	否
网络搜索	否	DuckDuckGo	是	WebSearch 工具	Google 搜索	否	通过 MCP
Sourcegraph	是	是	否	否	否	否	否
Git 集成	基础	是 (归属)	Ghost commit	通过 Hook	基础	深度 (自动提交)	基础
代码审查	否	否	是 (内置)	否	否	否	否
Diff 系统	统一 diff	Edit + MultiEdit	apply_patch (统一)	Edit (字符串替换)	未知	whole/diff/udiff	未知

4. 各类别最佳工具分析

类别	最佳工具	原因
沙箱机制	Codex	唯一具备操作系统级沙箱的工具（macOS 上的 Seatbelt，Linux 上的 Landlock）。自动模式下网络完全阻断。其他工具依赖权限提示。
LSP 集成	Crush	最深度的 LSP：诊断、引用、重启。多客户端管理器。View 工具自动附加诊断信息。OpenCode 仅有诊断功能。
MCP 支持	Crush / Goose	Crush：完整规范（工具+资源+提示词），3 种传输方式，状态机。Goose：扩展即 MCP 服务器。Codex 独特之处在于可作为 MCP 服务器。
工具广度	Crush	20+ 工具，包括后台任务、待办跟踪、多文件编辑、agentic fetch、DuckDuckGo、Sourcegraph。最全面的内置工具集。
多提供商	Codex	10+ 内置提供商，加上任何 OpenAI 兼容 API。开箱即用支持最广。
上下文管理	Gemini CLI	100 万 Token 窗口消除了压缩/摘要的需求。Claude Code 基于 Hook 的压缩机制是较小窗口中最精密的方案。
Hook/ 生命周期	Claude Code	14 个生命周期事件，支持完整拦截。其他工具无法企及。Crush 和 OpenCode 没有 Hook 支持。
子 Agent	Claude Code	Task 工具具备隔离上下文、模型选择（Opus/Sonnet/Haiku）、并行执行。Crush 有 agentic fetch。OpenCode 有只读子 Agent。
TUI 质量	Crush / OpenCode	两者均使用 Bubble Tea（Go 最佳 TUI 框架）。Crush 额外使用 Lip Gloss + Glamour 进行样式/Markdown 渲染。Codex 使用 Ratatui（Rust 对应方案）。
Git 集成	Aider	自动提交作为检查点，git 即会话管理。Codex 有 ghost commit。其他工具较为基础。
代码库理解	Aider	开创了 Tree-sitter 仓库地图。基于 AST 的上下文选择，支持 100+ 种语言。其他工具使用 grep/glob。
安全模型	Codex	操作系统级沙箱 + 审批模式 + 网络阻断。Crush 拥有最佳的基于权限的模型（多层、修改时间检查）。

类别	最佳工具	原因
技能/ 可扩展性	Claude Code	技能 + Hook + MCP = 最强可扩展性。Codex 有 SKILL.md + Automations。Crush 有技能目录。
后台执行	Crush	唯一支持自动转后台（60 秒阈值）、任务跟踪、任务终止的工具。 对开发服务器和长时间构建至关重要。
代码审查	Codex	内置审查，支持多种目标（未提交、分支、提交、自定义）。其他 CLI 没有原生代码审查。
跨仓库搜索	OpenCode / Crush	两者均集成 Sourcegraph。Crush 还有 DuckDuckGo。其他工具缺乏跨仓库能力。
循环检测	Crush	基于滑动窗口的 SHA-256 签名哈希。简单有效。Claude Code 通过 Hook 实现（更灵活但属于外部机制）。
提示词工程	Crush	Go 模板系统，编译时嵌入，git 上下文注入，技能 XML 序列化。最结构化的方案。
性能	Codex	Rust 原生二进制文件。启动和执行速度最快。Go 工具（OpenCode、Crush）位居第二。 TS/Python 工具最慢。
免费层	Gemini CLI	60 次请求/分钟，1K 次请求/天免费。其他工具无法提供同等的免费访问。

5. 对 cc-cli 的建议

基于本次分析，以下功能代表了新 CLI Agent 的最高价值实现方向：

必备功能（基本门槛）

- 多提供商抽象层** — 至少支持 5 个以上提供商（OpenAI、Anthropic、Google、通过 Ollama 的本地模型、OpenRouter）。采用类似 OpenCode/Crush 的提供商接口模式。
- MCP 客户端支持** — 完整规范：工具 + 资源 + 提示词。至少支持 stdio 传输，HTTP/SSE 用于远程服务器。参考 Crush 的状态机模式（Disabled→Starting→Connected→Error）。
- 权限系统** — 类似 Crush 的多层设计：命令黑名单、安全白名单、先读后编辑强制、修改时间检查。为高级用户添加 `--yolo` 等效选项。
- 会话持久化** — 基于 SQLite（类似 OpenCode/Codex）。存储消息、工具调用、元数据。支持会话恢复。
- 项目配置文件** — [AGENTS.md](#) 或等效方案，支持层级加载（全局 → 仓库 → 当前工作目录）。

高价值差异化功能

- LSP 集成** — 参考 Crush 的模型：诊断 + 引用 + 重启。多客户端管理器。文件查看时自动附加诊断。写入/补丁后运行诊断。这是一个重要的差异化优势——15 款工具中仅有 2 款具备此功能。
- Hook/生命周期系统** — Claude Code 的 14 事件系统是黄金标准。从 5-6 个核心事件起步：SessionStart、PreToolUse、PostToolUse、PreCompact、Stop。支持上下文注入和权限拦截。
- 后台任务管理** — Crush 的自动转后台模式（60 秒阈值）设计优雅。对开发服务器、构建、测试套件至关重要。包含 `job_output` 和 `job_kill`。
- 循环检测** — Crush 的 SHA-256 滑动窗口方案简单有效。10 步窗口，5 次重复阈值。
- 子 Agent 生成** — 隔离的上下文窗口，用于并行调研。只读工具子集确保安全。Claude Code 的 Task 工具是参考实现。

锦上添花（竞争优势）

- Sourcegraph 集成** — 跨仓库搜索对理解库使用模式非常有价值。OpenCode 和 Crush 均已具备。GraphQL API，实现简单。
- Codex 即 MCP 服务器模式** — 将 Agent 自身暴露为 MCP 服务器，实现元 Agent 架构。Codex 独有，创新价值高。
- 操作系统级沙箱** — 类似 Codex 的 Seatbelt（macOS）+ Landlock（Linux）。显著强于权限提示。实现复杂但安全性业界最佳。
- 内置代码审查** — Codex 的多目标审查（未提交、分支、提交）。自然的工作流集成。
- 技能/插件系统** — 基于文件系统，类似 Codex（[SKILL.md](#)）或 Crush（技能目录）。支持社区扩展。

- 16. **待办/任务跟踪** — Crush 的内置待办系统让用户可以了解多步操作的进度。实现简单，用户体验价值高。
- 17. **Agentic fetch** — Crush 的专用网络调研子 Agent，拥有独立工具集。在调研任务中优于原始 fetch。
- 18. **Ghost commit** — Codex 的不可见仓库快照，用于回滚。优雅的撤销机制。
- 19. **自动摘要** — OpenCode 和 Crush 均支持会话自动摘要。对于较小上下文窗口的管理至关重要。
- 20. **提示词模板系统** — Crush 的 Go 模板方案，编译时嵌入、git 上下文注入、技能序列化。结构化且易于维护。

架构建议

- **语言**：Go 或 Rust。Go 拥有更好的 TUI 生态（Bubble Tea）和更快的开发速度。Rust 提供最佳性能和沙箱能力。
- **TUI**：Bubble Tea（Go）或 Ratatui（Rust）。两者均成熟且文档完善。
- **数据库**：SQLite 用于会话、配置和状态。已被 OpenCode、Codex 和我们的 memory-tools 验证。
- **配置**：TOML（类似 Codex）或 JSON（类似 OpenCode）。分层：用户 → 项目。
- **Agent 循环**：单线程 ReAct，工具使用后继续。顺序工具执行，带权限检查。
- **提供商接口**：抽象提供商，包含 Generate() 和 Stream() 方法。函数式选项模式（类似 Crush 的 Hyper）。

6. 知识库参考

来源 ID

来源	ID	条目数	描述
OpenCode	262	10	架构、工具、Agent 循环、提供商
Crush	263	14	架构、工具、MCP、LSP、权限、提示词、独特功能
awesome-opencode	264	10	全景概览：Gemini CLI、Aider、Goose、Cline、Continue、Warp、Zed、Amp、Codebuff
Codex	265	8	架构、工具、沙箱、MCP、配置、技能、协议、独特功能

按主题分类的关键知识库条目 ID

OpenCode：

- 8368 — 架构概览
- 8369 — Agent 循环
- 8370 — 工具系统
- 8371 — Bash 工具
- 8373 — Patch 工具
- 8375 — Write 工具
- 8376 — Grep 工具

Crush：

- 8396 — 架构概览
- 8399 — 循环检测
- 8400 — Hyper 提供商
- 8401 — MCP 集成
- 8402 — 提示词系统
- 8403 — 后台任务
- 8404 — LSP 集成
- 8405 — Sourcegraph
- 8406 — 待办系统
- 8407 — 安全与权限
- 8409 — 独特功能对比

Codex:

- 8388 — 架构概览
- 8389 — 应用-服务器协议
- 8390 — 沙箱模型
- 8391 — 技能系统
- 8392 — 工具与 Agent 循环
- 8393 — 多模型与 MCP
- 8394 — 配置与会话
- 8395 — 独特功能及优劣势

全景 (awesome-opencode):

- 8378 — Aider
- 8379 — Goose
- 8380 — Gemini CLI
- 8381 — Cline
- 8382 — Continue.dev
- 8383 — Amp/Sourcegraph
- 8384 — Codebuff
- 8385 — Zed
- 8386 — Warp

知识库检索查询

```
mem_kb({action:"search", query:"OpenCode tools architecture"})
mem_kb({action:"search", query:"Crush MCP LSP permissions"})
mem_kb({action:"search", query:"Codex sandbox skills protocol"})
mem_kb({action:"search", query:"AI agent CLI comparison landscape"})
```

本报告基于 42 个知识库条目生成，涵盖 4 个已爬取的代码仓库及网络调研。所有数据已存储在 *memory-tools* 向量数据库中，供后续参考。